



Sistemas de Banco de Dados Vetoriais

Enzo Furegatti Spinella

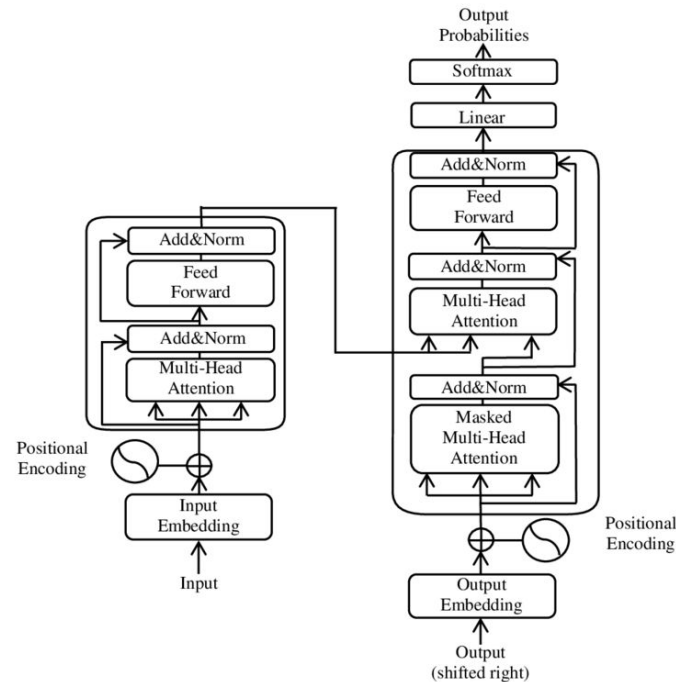
Ísis Ardisson Logullo

Luiz Fernando de Freitas Fernandes

Matheus Rodrigues da Silva Espaloor

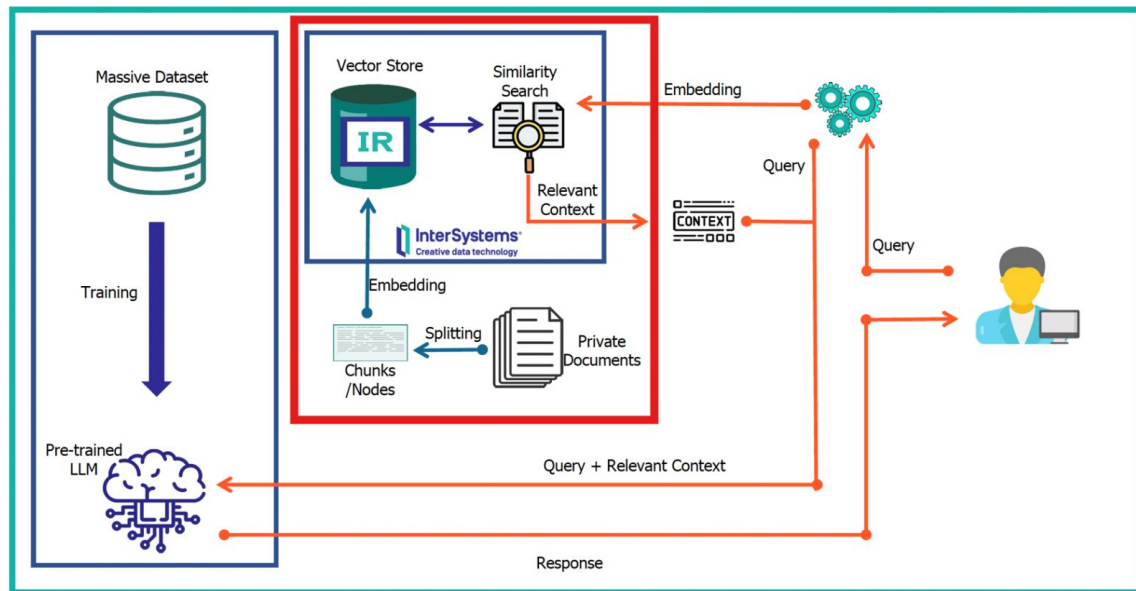
Contextualização e Motivação

- Boom dos LLMs
- Transformers
- Embeddings
- Deep Learning
- Limitações dos SGBDs tradicionais: maldição da dimensionalidade



Infraestrutura Vetorial

- RAG
- Busca Semântica Multimodal
- Sistemas de Recomendação de Próxima Geração





Fundamentos Matemáticos

- Produto escalar de Self-Attention
- Self-Attention de Múltiplas Cabeças
- Feed-Forward Networks (FFN)
- Codificação Posicional
- Considerações do Bloco Encoder-Decoder
- Métricas de Distância e Similaridade Vetorial
- Distância Euclidiana
- Similaridade de Cosseno
- Produto Interno
- Maldição da Dimensionalidade

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Q, K, V : São matrizes que contêm as representações vetoriais das palavras.

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

x : O vetor da palavra na posição atual após passar pela camada de atenção.

$\max(0, \cdot)$: Esta é a função de ativação ReLU (Rectified Linear Unit), que introduz não-linearidade no modelo. Se o valor for negativo, torna-se 0; se for positivo, mantém-se igual.

W_1, b_1, W_2, b_2 : São os pesos e bias das duas camadas lineares. A dimensão interna da FFN é $d_{ff} = 2048$, enquanto a de entrada/saída é $d_{model} = 512$.

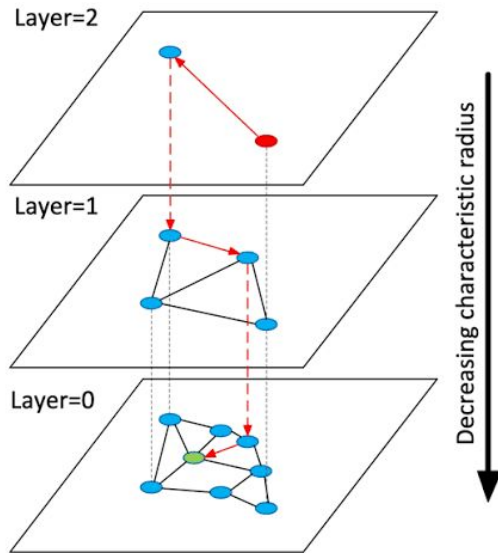


Algoritmos de Vizinho Mais Próximo Aproximado

- Acham (aproximadamente) um vetor do BD mais próximo de um vetor de entrada
- Maldição da dimensionalidade torna algoritmo exato inviável
- Essenciais em buscas em BDs vetoriais
- Dois tipos mais usados atualmente:
 - Mundo Pequeno Navegável Hierárquico (HNSW)
 - Inverted File Index com quantização de produto (IVFPQ)

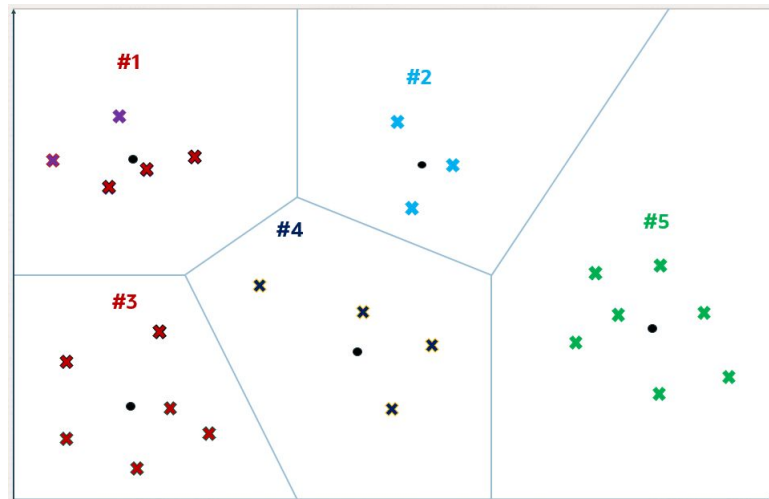
Mundo Pequeno Navegável Hierárquico (HNSW)

- Ideia é grafo navegável + skip list
- Constrói um grafo navegável com os vetores do BD e cria camadas com menos nós para agilizar busca (como skip lists)
- Buscas rápidas e precisas
- Alto uso de memória



Inverted File Index com Quantização de Produtos (IVFPQ)

- Inverted File Index: separa vetores do BD em clusters
 - Diminui espaço de possibilidades
- Quantização de Produtos: divide vetores em subvetores e quantiza cada um deles
 - Otimiza uso de memória e velocidade de varredura
- Baixo uso de recursos e alta escalabilidade
- Velocidade e precisão não muito altos



Análise Comparativa

—



Algoritmos de indexação e escalabilidade

Sistema	Algoritmo de indexação	Escalabilidade
Milvus	HNSW, IVF, ScaNN	Altamente distribuído e verticalmente escalável
Postgres	HNSW, IVFFlat	Vertical, depende da RAM e CPU de onde o PostgreSQL está rodando
Qdrant	HNSW customizado	Escalabilidade horizontal através de sharding e paralelização garantida pelo RUST



Algoritmos de Busca

Milvus	Filtragem dinâmica sobre os dados indexados e em stream
Postgres	Filtros relacionais e vetoriais integrados no SQL
Qdrant	Filtragem dinâmica em estágio [único durante a travessia do grafo



Vantagens e Desvantagens

Sistema	Vantagem	Desvantagem
Milvus	Altamente distribuído; paralelismo massivo e consistência dinâmica	Muito complexo de manter; múltiplos microsserviços; curva de aprendizado complicada
Postgres	ACID garantido pelo SGDB do PostgreSQL; integração nativa com dados relacionais	Limite de 2 mil dimensões; uso somente de CPU; a computação vetorial compete com transações SQL
Qdrant	Alta eficiência de memória; filtragem em estágio único; quantização nativa	Não tão eficiente quanto o Milvus; ferramenta nova

Exemplo

(Qdrant e Postgresql)

—



Ambiente e Metodologia

Ambiente Controlado: Contêineres Docker isolados rodando em Pop!_OS.

Massa de Dados: Subconjunto de 5.000 amostras do dataset *dair-ai/emotion*.

Vetorização: Modelo *all-MiniLM-L6-v2* gerando vetores de 384 dimensões.

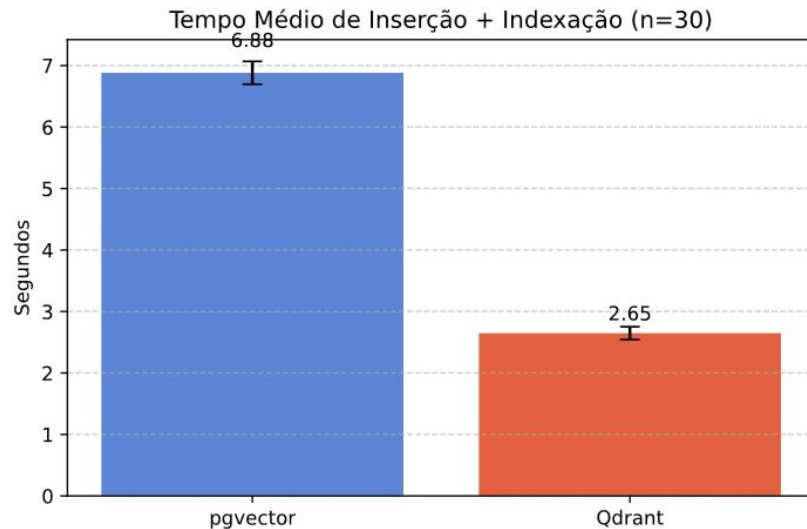
Rigor Estatístico: 30 repetições independentes (n=30) recriando os índices a cada ciclo.

Desempenho de Inserção

Qdrant na Liderança: Média de 2,65s contra 6,88s do pgvector.

A Força do Rust: Arquitetura nativa construída para processamento em blocos (*batches*).

O Gargalo Relacional: Sobrecarga transacional (ACID) do Postgres ao construir o grafo HNSW massivamente.

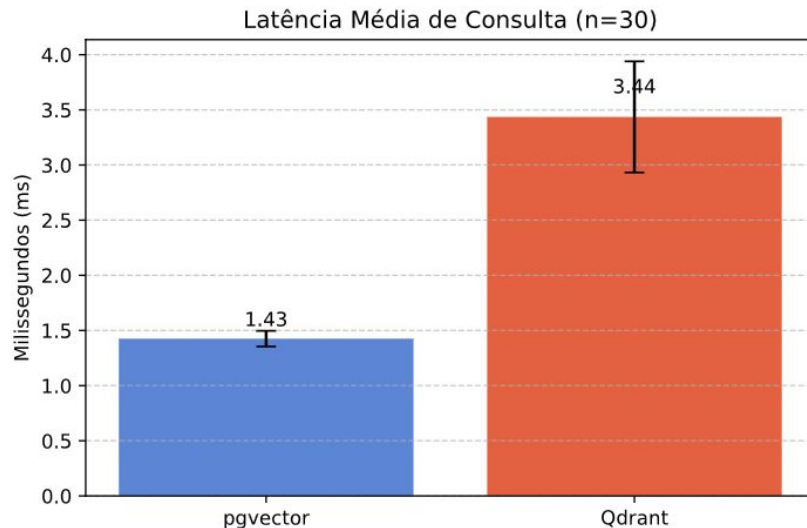


Latência de Busca

O Retorno do pgvector: Respostas em 1,43ms contra 3,44ms do Qdrant.

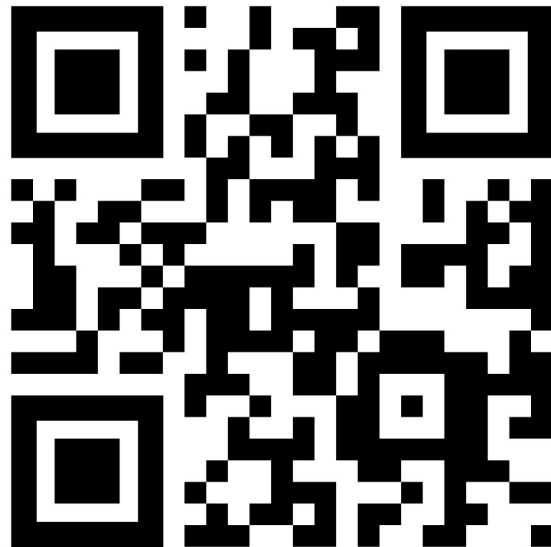
O Peso da Rede: A alta variância de erro do Qdrant reflete o custo de abrir conexões HTTP/REST.

Vantagem de Baixo Nível: Postgres utiliza conexão via *socket* binário persistente, eliminando atrasos do cliente.





Reprodutibilidade



<https://github.com/SQL-2026-Trabalho/ep2>

Transparência Científica: Todo o experimento segue os padrões do edital.

Código Aberto: Scripts Python e configuração do Docker disponíveis publicamente no GitHub.